

CRACKING *the* CODING INTERVIEW

189 PROGRAMMING QUESTIONS & SOLUTIONS



GAYLE LAAKMANN MCDOWELL

Author of Cracking the PM Interview and Cracking the Tech Career

6TH
EDITION

CRACKING
the
CODING INTERVIEW

6TH EDITION

ALSO BY GAYLE LAAKMANN McDOWELL

CRACKING THE PM INTERVIEW

HOW TO LAND A PRODUCT MANAGER JOB IN TECHNOLOGY

CRACKING THE TECH CAREER

INSIDER ADVICE ON LANDING A JOB AT GOOGLE, MICROSOFT, APPLE, OR ANY TOP TECH COMPANY

CRACKING *the* **CODING INTERVIEW**

6th Edition

189 Programming Questions and Solutions

GAYLE LAAKMANN MCDOWELL
Founder and CEO, CareerCup.com

CareerCup, LLC
Palo Alto, CA

CRACKING THE CODING INTERVIEW, SIXTH EDITION

Copyright © 2015 by CareerCup.

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means, including information storage and retrieval systems, without permission in writing from the author or publisher, except by a reviewer who may quote brief passages in a review.

Published by CareerCup, LLC, Palo Alto, CA. Compiled Feb 10, 2016.

For more information, contact support@careercup.com

978-0-9847828-5-7 (ISBN 13)

*For Davis and Tobin,
and all the things that bring us joy in life.*

Introduction	2
I. The Interview Process	4
Why?	4
How Questions are Selected	6
It's All Relative	7
Frequently Asked Questions	7
II. Behind the Scenes	8
The Microsoft Interview	9
The Amazon Interview	10
The Google Interview	10
The Apple Interview	11
The Facebook Interview	12
The Palantir Interview	13
III. Special Situations	15
Experienced Candidates	15
Testers and SOETs	15
Product (and Program) Management	15
Dev Lead and Managers	17
Startups	18
Acquisitions and AcquiHires	19
For Interviewers	21
IV. Before the Interview	26
Getting the Right Experience	26
Writing a Great Resume	27
Preparation Map	30
V. Behavioral Questions	32
Interview Preparation Grid	32
Know Your Technical Projects	33
Responding to Behavioral Questions	34
So, tell me about yourself.....	36
VI. Big O	38
An Analogy	38
Time Complexity	38
Space Complexity	40
Drop the Constants	41
Drop the Non-Dominant Terms	42

Multi-Part Algorithms: Add vs. Multiply	42
Amortized Time	43
Log N Runtimes	44
Recursive Runtimes	44
Examples and Exercises	45
VII. Technical Questions	60
How to Prepare	60
What You Need to Know	60
Walking Through a Problem	62
Optimize & Solve Technique #1: Look for BJD	67
Optimize & Solve Technique #2: DIY (Do It Yourself)	69
Optimize & Solve Technique #3: Simplify and Generalize	71
Optimize & Solve Technique #4: Base Case and Build	71
Optimize & Solve Technique #5: Data Structure Brainstorm	72
Best Conceivable Runtime (BCR)	72
Handling Incorrect Answers	76
When You've Heard a Question Before	76
The "Perfect" Language for Interviews	76
What Good Coding Looks Like	77
Don't Give Up!	81
VIII. The Offer and Beyond	82
Handling Offers and Rejection	82
Evaluating the Offer	83
Negotiation	84
On the Job	85
IX. Interview Questions	87
Data Structures	88
Chapter 1 Arrays and Strings	88
Hash Tables	88
ArrayList & Resizable Arrays	89
StringBuilder	89
Chapter 2 Linked Lists	92
Creating a Linked List	92
Deleting a Node from a Singly Linked List	93
The "Runner" Technique	93
Recursive Problems	93

Chapter 3 Stacks and Queues	95
<i>Implementing a Stack</i>	96
<i>Implementing a Queue</i>	97
Chapter 4 Trees and Graphs	100
<i>Types of Trees</i>	100
<i>Binary Tree Traversal</i>	103
<i>Binary Heaps (Min-Heaps and Max-Heaps)</i>	103
<i>Tries (Prefix Trees)</i>	105
<i>Graphs</i>	105
<i>Graph Search</i>	107
Concepts and Algorithms	112
Chapter 5 Bit Manipulation	112
<i>Bit Manipulation By Hand</i>	112
<i>Bit Flags and Tricks</i>	112
<i>Two's Complement and Negative Numbers</i>	113
<i>Arithmetic vs. Logical Right Shift</i>	113
<i>Common Bit Tasks: Getting and Setting</i>	114
Chapter 6 Math and Logic Puzzles	117
<i>Prime Numbers</i>	117
<i>Probability</i>	119
<i>Start Talking</i>	121
<i>Develop Rules and Patterns</i>	121
<i>Worst Case Shifting</i>	122
<i>Algorithm Approaches</i>	122
Chapter 7 Object-Oriented Design	125
<i>How to Approach</i>	125
<i>Design Patterns</i>	126
Chapter 8 Recursion and Dynamic Programming	130
<i>How to Approach</i>	130
<i>Recursive vs. Iterative Solutions</i>	131
<i>Dynamic Programming & Memoization</i>	131
Chapter 9 System Design and Scalability	137
<i>Handling the Questions</i>	137
<i>Design: Step-By-Step</i>	138
<i>Algorithms that Scale: Step-By-Step</i>	139
<i>Key Concepts</i>	140

Considerations	142
There is no "perfect" system.	143
Example Problem.	143
Chapter 10 Sorting and Searching	146
Common Sorting Algorithms	146
Searching Algorithms.	149
Chapter 11 Testing.	152
What the Interviewer is Looking For	152
Testing a Real World Object	153
Testing a Piece of Software	154
Testing a Function	155
Troubleshooting Questions	156
Knowledge Based	158
Chapter 12 C and C++.	158
Classes and Inheritance.	158
Constructors and Destructors.	159
Virtual Functions	159
Virtual Destructor	160
Default Values.	161
Operator Overloading.	161
Pointers and References	162
Templates.	163
Chapter 13 Java.	165
How to Approach.	165
Overloading vs. Overriding	165
Collection Framework.	166
Chapter 14 Databases.	169
SQL Syntax and Variations.	169
Denormalized vs. Normalized Databases.	169
SQL Statements.	169
Small Database Design	171
Large Database Design.	172
Chapter 15 Threads and Locks	174
Threads in Java	174
Synchronization and Locks	176
Deadlocks and Deadlock Prevention	179

Additional Review Problems	181
Chapter 16 Moderate	187
Chapter 17 Hard	186
X. Solutions	191
Data Structures	192
Concepts and Algorithms	276
Knowledge Based	422
Additional Review Problems	462
XI. Advanced Topics	628
Useful Math	629
Topological Sort	632
Dijkstra's Algorithm	633
Hash Table Collision Resolution	636
Rabin-Karp Substring Search	636
AVL Trees	637
Red-Black Trees	639
MapReduce	642
Additional Studying	644
XII. Code Library	645
HashMapList<I, E>	646
TreeNode (Binary Search Tree)	647
LinkedListNode (LinkedList)	649
Trie & TrieNode	649
XIII. Hints	652
Hints for Data Structures	653
Hints for Concepts and Algorithms	662
Hints for Knowledge-Based Questions	676
Hints for Additional Review Problems	679
XIV. About the Author	696

Join us at www.CrackingTheCodingInterview.com to download the complete solutions, contribute or view solutions in other programming languages, discuss problems from this book with other readers, ask questions, report issues, view this book's errata, and seek additional advice.

Dear Reader,

Let's get the introductions out of the way.

I am not a recruiter. I am a software engineer. And as such, I know what it's like to be asked to whip up brilliant algorithms on the spot and then write flawless code on a whiteboard. I know because I've been asked to do the same thing—in interviews at Google, Microsoft, Apple, and Amazon, among other companies.

I also know because I've been on the other side of the table, asking candidates to do this. I've combed through stacks of resumes to find the engineers who I thought might be able to actually pass these interviews. I've evaluated them as they solved—or tried to solve—challenging questions. And I've debated in Google's Hiring Committee whether a candidate did well enough to merit an offer. I understand the full hiring cycle because I've been through it all, repeatedly.

And you, reader, are probably preparing for an interview, perhaps tomorrow, next week, or next year. I am here to help you solidify your understanding of computer science fundamentals and then learn how to apply those fundamentals to crack the coding interview.

The 6th edition of *Cracking the Coding Interview* updates the 5th edition with 70% more content: additional questions, revised solutions, new chapter introductions, more algorithm strategies, hints for all problems, and other content. Be sure to check out our website, CrackingTheCodingInterview.com, to connect with other candidates and discover new resources.

I'm excited for you and for the skills you are going to develop. Thorough preparation will give you a wide range of technical and communication skills. It will be well worth it, no matter where the effort takes you!

I encourage you to read these introductory chapters carefully. They contain important insight that just might make the difference between a "hire" and a "no hire."

And remember—interviews are hard! In my years of interviewing at Google, I saw some interviewers ask "easy" questions while others ask harder questions. But you know what? Getting the easy questions doesn't make it any easier to get the offer. Receiving an offer is not about solving questions flawlessly (very few candidates do!). Rather, it is about answering questions better than other candidates. So don't stress out when you get a tricky question—everyone else probably thought it was hard too. It's okay to not be flawless.

Study hard, practice—and good luck!

Gayle L. McDowell

Founder/CEO, CareerCup.com

Author of *Cracking the PM Interview* and *Cracking the Tech Career*

Something's Wrong

We walked out of the hiring meeting frustrated—again. Of the ten candidates we reviewed that day, none would receive offers. Were we being too harsh, we wondered?

I, in particular, was disappointed. We had rejected one of my candidates. A former student. One I had referred. He had a 3.73 GPA from the University of Washington, one of the best computer science schools in the world, and had done extensive work on open source projects. He was energetic. He was creative. He was sharp. He worked hard. He was a true geek in all the best ways.

But I had to agree with the rest of the committee: the data wasn't there. Even if my emphatic recommendation could sway them to reconsider, he would surely get rejected in the later stages of the hiring process. There were just too many red flags.

Although he was quite intelligent, he struggled to solve the interview problems. Most successful candidates could fly through the first question, which was a twist on a well-known problem, but he had trouble developing an algorithm. When he came up with one, he failed to consider solutions that optimized for other scenarios. Finally, when he began coding, he flew through the code with an initial solution, but it was riddled with mistakes that he failed to catch. Though he wasn't the worst candidate we'd seen by any measure, he was far from meeting the "bar." Rejected.

When he asked for feedback over the phone a couple of weeks later, I struggled with what to tell him. Be smarter? No. I knew he was brilliant. Be a better coder? No, his skills were on par with some of the best I'd seen.

Like many motivated candidates, he had prepared extensively. He had read K&R's classic C book, and he'd reviewed CLRS' famous algorithms textbook. He could describe in detail the myriad of ways of balancing a tree, and he could do things in C that no sane programmer should ever want to do.

I had to tell him the unfortunate truth: those books aren't enough. Academic books prepare you for fancy research, and they will probably make you a better software engineer, but they're not sufficient for interviews. Why? I'll give you a hint. Your interviewers haven't seen red-black trees since they were in school either.

To crack the coding interview, you need to prepare with real interview questions. You must practice on real problems and learn their patterns. It's about developing a fresh algorithm, not memorizing existing problems.

Cracking the Coding Interview is the result of my first-hand experience interviewing at top companies and later coaching candidates through these interviews. It is the result of hundreds of conversations with candidates. It is the result of the thousands of questions contributed by candidates and interviewers. And it's the result of seeing so many interview questions from so many firms. Enclosed in this book are 189 of the best interview questions, selected from thousands of potential problems.

My Approach

The focus of *Cracking the Coding Interview* is algorithm, coding, and design questions. Why? Because while you can and will be asked behavioral questions, the answers will be as varied as your resume. Likewise, while many firms will ask so-called "trivia" questions (e.g., "What is a virtual function?"), the skills developed through practicing these questions are limited to very specific bits of knowledge. The book will briefly touch on some of these questions to show you what they're like, but I have chosen to allocate space to areas where there's more to learn.

My Passion

Teaching is my passion. I love helping people understand new concepts and giving them tools to help them excel in their passions.

My first official experience teaching was in college at the University of Pennsylvania, when I became a teaching assistant for an undergraduate computer science course during my second year. I went on to TA for several other courses, and I eventually launched my own computer science course there, focused on hands-on skills.

As an engineer at Google, training and mentoring new engineers were some of the things I enjoyed most. I even used my “20% time” to teach two computer science courses at the University of Washington.

Now, years later, I continue to teach computer science concepts, but this time with the goal of preparing engineers at startups for their acquisition interviews. I’ve seen their mistakes and struggles, and I’ve developed techniques and strategies to help them combat those very issues.

Cracking the Coding Interview, *Cracking the PM Interview*, *Cracking the Tech Career*, and *CareerCup* reflect my passion for teaching. Even now, you can often find me “hanging out” at CareerCup.com, helping users who stop by for assistance.

Join us.

Gayle L. McDowell

The Interview Process

At most of the top tech companies (and many other companies), algorithm and coding problems form the largest component of the interview process. Think of these as problem-solving questions. The interviewer is looking to evaluate your ability to solve algorithmic problems you haven't seen before.

Very often, you might get through only one question in an interview. Forty-five minutes is not a long time and it's difficult to get through several different questions in that time frame.

You should do your best to talk out loud throughout the problem and explain your thought process. Your interviewer might jump in sometimes to help you, let them. It's normal and doesn't really mean that you're doing poorly. (That said, of course not needing hints is even better.)

At the end of the interview, the interviewer will walk away with a gut feel for how you did. A numeric score might be assigned to your performance, but it's not actually a quantitative assessment. There's no chart that says how many points you get for different things. It just doesn't work like that.

Rather, your interviewer will make an assessment of your performance, usually based on the following:

- **Analytical skills:** Did you need much help solving the problem? How optimal was your solution? How long did it take you to arrive at a solution? If you had to design/architect a new solution, did you structure the problem well and think through the tradeoffs of different decisions?
- **Coding skills:** Were you able to successfully translate your algorithm to reasonable code? Was it clean and well-organized? Did you think about potential errors? Did you use good style?
- **Technical knowledge / Computer Science fundamentals:** Do you have a strong foundation in computer science and the relevant technologies?
- **Experience:** Have you made good technical decisions in the past? Have you built interesting, challenging projects? Have you shown drive, initiative, and other important factors?
- **Culture fit / Communication skills:** Do your personality and values fit with the company and team? Did you communicate well with your interviewer?

The weighting of these areas will vary based on the question, interviewer, role, team, and company. In a standard algorithm question, it might be almost entirely the first three of those.

► Why?

This is one of the most common questions candidates have as they get started with this process. Why do things this way? After all,

1. Lots of great candidates don't do well in these sorts of interviews.

2. You could look up the answer if it did ever come up.
3. You rarely have to use data structures such as binary search trees in the real world. If you did need to, you could surely learn it.
4. Whiteboard coding is an artificial environment. You would never code on the whiteboard in the real world, obviously.

These complaints aren't without merit. In fact, I agree with all of them, at least in part.

At the same time, there is reason to do things this way for some—not all—positions. It's not important that you agree with this logic, but it is a good idea to understand why these questions are being asked. It helps offer a little insight into the interviewer's mindset.

False negatives are acceptable.

This is sad (and frustrating for candidates), but true.

From the company's perspective, it's actually acceptable that some good candidates are rejected. The company is out to build a great set of employees. They can accept that they miss out on some good people. They'd prefer not to, of course, as it raises their recruiting costs. It is an acceptable tradeoff though, provided they can still hire enough good people.

They're far more concerned with false positives: people who do well in an interview but are not in fact very good.

Problem-solving skills are valuable.

If you're able to work through several hard problems (with some help, perhaps), you're probably pretty good at developing optimal algorithms. You're smart.

Smart people tend to do good things, and that's valuable at a company. It's not the only thing that matters, of course, but it is a really good thing.

Basic data structure and algorithm knowledge is useful.

Many interviewers would argue that basic computer science knowledge is, in fact, useful. Understanding trees, graphs, lists, sorting, and other knowledge does come up periodically. When it does, it's really valuable.

Could you learn it as needed? Sure. But it's very difficult to know that you should use a binary search tree if you don't know of its existence. And if you do know of its existence, then you pretty much know the basics.

Other interviewers justify the reliance on data structures and algorithms by arguing that it's a good "proxy." Even if the skills wouldn't be that hard to learn on their own, they say it's reasonably well-correlated with being a good developer. It means that you've either gone through a computer science program (in which case you've learned and retained a reasonably broad set of technical knowledge) or learned this stuff on your own. Either way, it's a good sign.

There's another reason why data structure and algorithm knowledge comes up, because it's hard to ask problem-solving questions that don't involve them. It turns out that the vast majority of problem-solving questions involve some of these basics. When enough candidates know these basics, it's easy to get into a pattern of asking questions with them.

Whiteboards let you focus on what matters.

It's absolutely true that you'd struggle with writing perfect code on a whiteboard. Fortunately, your interviewer doesn't expect that. Virtually everyone has some bugs or minor syntactical errors.

The nice thing about a whiteboard is that, in some ways, you can focus on the big picture. You don't have a compiler, so you don't need to make your code compile. You don't need to write the entire class definition and boilerplate code. You get to focus on the interesting, "meaty" parts of the code—the function that the question is really all about.

That's not to say that you should just write pseudocode or that correctness doesn't matter. Most interviewers aren't okay with pseudocode, and fewer errors are better.

Whiteboards also tend to encourage candidates to speak more and explain their thought process. When a candidate is given a computer, their communication drops substantially.

But it's not for everyone or every company or every situation.

The above sections are intended to help you understand the thought process of the company.

My personal thoughts? For the right situation, when done well, it's a reasonable judge of someone's problem-solving skills. In that people who do well tend to be fairly smart.

However, it's often not done very well. You have bad interviewers or people who just ask bad questions.

It's also not appropriate for all companies. Some companies should value someone's prior experience more or need skills with particular technologies. These sorts of questions don't put much weight on that.

It also won't measure someone's work ethic or ability to focus. Then again, almost no interview process can really evaluate this.

This is not a perfect process by any means, but what is? All interview processes have their downsides.

I'll leave you with this: it is what it is, so let's do the best we can with it.

► How Questions are Selected

Candidates frequently ask what the "recent" interview questions are at a specific company. Just asking this question reveals a fundamental misunderstanding of where questions come from.

At the vast majority of companies, there are no lists of what interviewers should ask. Rather, each interviewer selects their own questions.

Since it's somewhat of a "free for all" as far as questions, there's nothing that makes a question a "recent Google interview question" other than the fact that some interviewer who happens to work at Google just so happened to ask that question recently.

The questions asked this year at Google do not really differ from those asked three years ago. In fact, the questions asked at Google generally don't differ from those asked at similar companies (Amazon, Facebook, etc.).

There are some broad differences across companies. Some companies focus on algorithms (often with some system design worked in), and others really like knowledge-based questions. But within a given category of question, there is little that makes it "belong" to one company instead of another. A Google algorithm question is essentially the same as a Facebook algorithm question.

► It's All Relative

If there's no grading system, how are you evaluated? How does an interviewer know what to expect of you?

Good question. The answer actually makes a lot of sense once you understand it.

Interviewers assess you relative to other candidates on that same question by the same interviewer. It's a relative comparison.

For example, suppose you came up with some cool new brainteaser or math problem. You ask your friend Alex the question, and it takes him 50 minutes to solve it. You ask Bella and she takes 50 minutes. Chris is never able to solve it. Dexter takes 15 minutes, but you had to give him some major hints and he probably would have taken far longer without them. Ellie takes 10—and comes up with an alternate approach you weren't even aware of. Fred takes 35 minutes.

You'll walk away saying, "Wow, Ellie did really well. I bet she's pretty good at math." (Of course, she could have just gotten lucky. And maybe Chris got unlucky. You might ask a few more questions just to really make sure that it wasn't good or bad luck.)

Interview questions are much the same way. Your interviewer develops a feel for your performance by comparing you to other people. It's not about the candidates she's interviewing *this week*. It's about all the candidates that she's ever asked this question to.

For this reason, getting a hard question isn't a bad thing. When it's harder for you, it's harder for everyone. It doesn't make it any less likely that you'll do well.

► Frequently Asked Questions

I didn't hear back immediately after my interview. Am I rejected?

No. There are a number of reasons why a company's decision might be delayed. A very simple explanation is that one of your interviewers hasn't provided their feedback yet. Very, very few companies have a policy of not responding to candidates they reject.

If you haven't heard back from a company within 3–5 business days after your interview, check in (politely) with your recruiter.

Can I re-apply to a company after getting rejected?

Almost always, but you typically have to wait a bit (6 months to a 1 year). Your first bad interview usually won't affect you too much when you re-interview. Lots of people get rejected from Google or Microsoft and later get offers from them.



Behind the Scenes

Most companies conduct their interviews in very similar ways. We will offer an overview of how companies interview and what they're looking for. This information should guide your interview preparation and your reactions during and after the interview.

Once you are selected for an interview, you usually go through a screening interview. This is typically conducted over the phone. College candidates who attend top schools may have these interviews in-person.

Don't let the name fool you; the "screening" interview often involves coding and algorithms questions, and the bar can be just as high as it is for in-person interviews. If you're unsure whether or not the interview will be technical, ask your recruiting coordinator what position your interviewer holds for what the interview might cover. An engineer will usually perform a technical interview.

Many companies have taken advantage of online synchronized document editors, but others will expect you to write code on paper and read it back over the phone. Some interviewers may even give you "homework" to solve after you hang up the phone or just ask you to email them the code you wrote.

You typically do one or two screening interviews before being brought on-site.

In an on-site interview round, you usually have 3 to 6 in-person interviews. One of these is often over lunch. The lunch interview is usually not technical, and the interviewer may not even submit feedback. This is a good person to discuss your interests with and to ask about the company culture. Your other interviews will be mostly technical and will involve a combination of coding, algorithms, design/architecture, and behavioral/experience questions.

The distribution of questions between the above topics varies between companies and even teams due to company priorities, size, and just pure randomness. Interviewers are often given a good deal of freedom in their interview questions.

After your interview, your interviewers will provide feedback in some form. In some companies, your interviewers meet together to discuss your performance and come to a decision. In other companies, interviewers submit a recommendation to a hiring manager or hiring committee to make a final decision. In some companies, interviewers don't even make the decision; their feedback goes to a hiring committee to make a decision.

Most companies get back after about a week with next steps (offer, rejection, further interviews, or just an update on the process). Some companies respond much sooner (sometimes same day) and others take much longer.

If you have waited more than a week, you should follow up with your recruiter. If your recruiter does not respond, this does *not* mean that you are rejected (at least not at any major tech company, and almost any

other company). Let me repeat that again: not responding indicates nothing about your status. The intention is that all recruiters should tell candidates once a final decision is made.

Delays can and do happen. Follow up with your recruiter if you expect a delay, but be respectful when you do. Recruiters are just like you. They get busy and forgetful too.

► The Microsoft Interview

Microsoft wants smart people. Geeks. People who are passionate about technology. You probably won't be tested on the ins and outs of C++ APIs, but you will be expected to write code on the board.

In a typical interview, you'll show up at Microsoft at some time in the morning and fill out initial paperwork. You'll have a short interview with a recruiter who will give you a sample question. Your recruiter is usually there to prep you, not to grill you on technical questions. If you get asked some basic technical questions, it may be because your recruiter wants to ease you into the interview so that you're less nervous when the "real" interview starts.

Be nice to your recruiter. Your recruiter can be your biggest advocate, even pushing to re-interview you if you stumbled on your first interview. They can fight for you to be hired—or not!

During the day, you'll do four or five interviews, often with two different teams. Unlike many companies, where you meet your interviewers in a conference room, you'll meet with your Microsoft interviewers in their office. This is a great time to look around and get a feel for the team culture.

Depending on the team, interviewers may or may not share their feedback on you with the rest of the interview loop.

When you complete your interviews with a team, you might speak with a hiring manager (often called the "as app," short for "as appropriate"). If so, that's a great sign! It likely means that you passed the interviews with a particular team. It's now down to the hiring manager's decision.

You might get a decision that day, or it might be a week. After one week of no word from HR, send a friendly email asking for a status update.

If your recruiter isn't very responsive, it's because she's busy, not because you're being silently rejected.

Definitely Prepare:

"Why do you want to work for Microsoft?"

In this question, Microsoft wants to see that you're passionate about technology. A great answer might be, "I've been using Microsoft software as long as I can remember, and I'm really impressed at how Microsoft manages to create a product that is universally excellent. For example, I've been using Visual Studio recently to learn game programming, and its APIs are excellent." Note how this shows a passion for technology!

What's Unique:

You'll only reach the hiring manager if you've done well, so if you do, that's a great sign!

Additionally, Microsoft tends to give teams more individual control, and the product set is diverse. Experiences can vary substantially across Microsoft since different teams look for different things.